

Universidad Técnica Nacional

Lic. Roosevelt Reyes

Propuesta de Proyecto

Integrantes:

Paula Nicole Mora Gomez,
Angelica Jazmin Esquivel Mairena

Asistencia de casos/citas

Periodo lectivo

Cuatrimestre III

Informe: Solución de Internacionalización (i18n) para CopyVet

Este documento resume la investigación e implementación de una solución de internacionalización (i18n) para la aplicación web CopyVet. El objetivo fue soportar, de manera robusta y eficiente, al menos dos idiomas: español (es) e inglés (en), cubriendo selección de librería, configuración del entorno, formatos de traducción, implementación en frontend, selector de idioma y persistencia de la preferencia de idioma.

1. Selección de librería/framework de I18N

La elección de i18next y sus extensiones se basó en que ofrecen una integración directa y estable con React, permitiendo implementar internacionalización sin agregar complejidad al proyecto. Además, estas librerías proporcionan soporte nativo para detección automática de idioma, manejo de namespaces y carga modular de traducciones, lo cual se ajusta perfectamente a la estructura de CopyVet. Su uso generalizado en la industria, la facilidad para organizar archivos JSON por módulos y la disponibilidad de una API clara hicieron que esta opción fuera la más adecuada para lograr una solución escalable, mantenible y rápida de implementar.

Librerías investigadas: i18next, react-i18next, i18next-browser-languagedetector.

Las librerías que se utilizaron en la aplicación fueron : **i18next** junto con **react-i18next** y **i18next-browser-languagedetector**.

Justificación:

- **Compatibilidad con React** (react-i18next ofrece hooks y HOCs adecuados).
- **Madurez:** i18next es una de las soluciones más usadas en la comunidad JavaScript con amplio soporte.
- **Rendimiento:** carga diferencial por namespaces y lazy-loading de recursos para evitar cargas iniciales pesadas.
- **Formatos soportados:** JSON de forma nativa con una fácil integración con formatos estándar y procesos de localización.
- **Facilidad de integración:** API clara (useTranslation, i18n.changeLanguage), soporte para interpolación, pluralización y caching.

3. Configuración del entorno de traducción

Estructura modular recomendada para index.js (configuración inicial de i18n):

Características clave:

- Registro de namespaces por módulo (**common, veterinary, ticket, auth, category, pet**).
- Uso de i18next-browser-languagedetector para detectar la preferencia del usuario.
- FallbackLang configurado a 'es' (**español**) para el comportamiento cuando falten claves.
- **Caches y orden de detección:** localStorage, navigator, HTML tag.

```
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
import LanguageDetector from 'i18next-browser-languagedetector';

import esCommon from './locales/es/common.json';
import enCommon from './locales/en/common.json';
```

```
i18n
  .use(LanguageDetector)
  .use(initReactI18next)
  .init({
    resources: {
      es: { common: esCommon, /* ... */ },
      en: { common: enCommon, /* ... */ },
    },
    fallbackLng: 'es',
    defaultNS: 'common',
    ns: ['common', 'ticket', 'veterinary', 'auth', 'category', 'pet'],
    detection: { order: ['localStorage', 'navigator', 'htmlTag'], caches: ['localStorage'] },
    interpolation: { escapeValue: false }
  });
```

4. Formatos de archivos de traducción

Se utilizaron archivos JSON por idioma y namespace, organizados en:

src/i18n/locales/{es,en}/{namespace}.json

Ejemplo:

src/i18n/locales/es/common.json

src/i18n/locales/en/common.json

Ventajas al utilizar esto:

- Fácil edición y control de versiones (git).
- Compatibilidad con herramientas de traducción y flujos CI/CD.
- Permite lazy-loading por namespace y una estructura lógica por módulos.

5. Implementación en la Interfaz de Usuario (Frontend)

Integración con componentes React mediante el hook useTranslation de react-i18next. Todos los elementos textuales (etiquetas, títulos, botones, mensajes, placeholders) deben invocar la función t() con la clave adecuada.

Ejemplo en un componente en CopyVet:

```
import { useTranslation } from "react-i18next";
const { t, i18n } = useTranslation();

return (
  <Button>{t("common:buttons.save")}</Button>
);
```

Recomendaciones de implementación que se utilizaron en nuestra pagina:

- Siempre usar claves con namespace (namespace:key) para evitar colisiones.
- Extraer textos de componentes a los JSON cuando no provengan de la base de datos.
- Marcar placeholders y mensajes dinámicos para interpolación (ej. 'Hola, {{name}}').
- Añadir pruebas básicas que verifican renderizado en ambos idiomas.

6. Mecanismo de selección de idioma

Implementar un componente **LanguageSwitcher** que permita seleccionar el idioma visible al usuario.

Ejemplo:

```
import { useTranslation } from 'react-i18next';
function LanguageSwitcher() {
  const { i18n } = useTranslation();
  return (
    <div>
      <button onClick={() => i18n.changeLanguage('es')}>ES</button>
      <button onClick={() => i18n.changeLanguage('en')}>EN</button>
    </div>
  );
}
```

```

}
const { i18n, t } = useTranslation();
<Button onClick={() => i18n.changeLanguage('en')}>
  {t("common:language.changeLanguage")}
</Button>

```

7. Persistencia de la selección de idioma

La preferencia se persiste en localStorage mediante i18next-browser-languagedetector (configuración de caches:['localStorage']). Esto garantiza que la selección se mantenga entre sesiones y cambios de página sin necesidad de backend.

Alternativas/Mejoras que se pueden considerar:

- Guardar la preferencia en el perfil de usuario (requiere endpoint en backend) para sincronizar entre dispositivos.
- Agregar expiración o lógica avanzada si se requiere soporte multi-tenant.

8. Alcance: qué traducir y qué no

Se traducirán todas las cadenas estáticas de la UI: etiquetas, títulos, botones, notificaciones, mensajes de validación, placeholders, textos de ayuda y menús. No se traducirán contenidos cargados desde base de datos o entradas de usuarios como la respuesta en el mismo momento.

9. Validación y pruebas

- Verificar que todos los componentes claves muestren traducciones en ambos idiomas.
- Pruebas automatizadas (render snapshots) para ambos idiomas en componentes críticos.
- Prueba manual del selector de idioma y persistencia en distintos navegadores.
- Validar pluralización e interpolación con casos de borde.

10. Procedimiento: añadir nuevas claves y nuevos idiomas

Añadir una nueva clave:

1. Agregar la clave en el JSON del namespace correspondiente en ambos idiomas.
2. Usar t('namespace:clave') en los componentes.

Agregar un nuevo idioma (ej. portugués):

1. Crear carpeta `src/i18n/locales/pt/` y agregar archivos `{namespace}.json`.
2. Importar y registrar los recursos en la configuración de `i18n` (`index.js`) o usar `lazy-loading`.
3. Actualizar el `LanguageSwitcher` para incluir la nueva opción.

}

11. Anexos: Snippets y ejemplos adicionales

- Ejemplo JSON (`common.json`):

```
{
  "buttons": {
    "save": "Guardar",
    "cancel": "Cancelar"
  },
  "messages": {
    "loading": "Cargando..."
  }
}
```

- Ejemplo para pluralización:

```
"ticketCount": "Tienes {{count}} tiquete{{count, plural, one {} other {s}}}"
```

- Ejemplo de `lazy-loading` por namespace (pseudo-código):

```
i18n.loadNamespaces(['veterinary']).then(() => { /* render component */ });
```